

computable adjudication and agentic test harness

caaat-harness

ca&at-harness

.c&at-harness

c@t-harness

Please be introduced to a cat who acquires things, for whatever purpose — maybe somebody knows.

Scope: the platform — skills, schemas, processes, the MCP server and the pipeline. Not what a folio IS (that is folio-assist-core), and not any folio's own content, which lives in its own repository.

- cat-harness, in the source

Still to be split out of this repository — issue #223.

- RTFM — the cat-harness docs

The agent–user interaction model, and what this harness is for.

- The work plan is where you say what you are doing

Claim a bean before you work, so a sibling session does not pick up the same item.

- Make your first folio

This repository is the platform; your content lives in its own.

- Know which kind of thing you are writing

A document is structured prose; a paper adds the block kinds whose assertion is a formal claim.

- The documentation you will never read

All of it. The honest expectation is that you arrive from a search engine when something breaks.

✖ View

✎ Edit

The knowledge graph is four sub-graphs, and an instance declares which of them it holds.

Content is what somebody authors and a reader reads — this layer's subject. **Skills** are the instructions an agent follows to do the work. **Processes** are the BPMN diagrams that say who does what, in which order, and where the gates are — they are executable, not illustrations. **Tools** are what an agent can call.

Getting anything in is **acquisition**: unstructured and semi-structured sources arrive through `uploads/` or through the conversation, and are ingested into `library/`. It is a step rather than a graph kind of its own — there is no fifth directory to look for.

Scope: the content model — what a folio IS. Not the skills and tools that operate on it, and not any particular folio's chapters.

- **Content**

A document is structured prose; a paper adds the block kinds whose assertion is a formal claim.

- **Skills**

Every skill and role, and how they fit together.

- **Processes**

The BPMN diagrams, which are the source of truth rather than a picture of it.

- **Tools**

What an agent can call, served over MCP.

- **Acquisition**

How a source gets from uploads/ into library/.

- **RTFM — folio-assist-core**

Its scope, in and out, and why it declares no directories yet.

⚙ View

✎ Edit

The graph an agent reads before it knows what this repository is.

Scope: phase one of two, and valid with NOTHING installed — no skills, no knowledge graph, no MCP, no beans. One process lives here and only one. Everything else is composed on top of it, and nothing here may import from the layer above.

- **bootstrap, in the source**

One process lives here and only one: workflows/initialize-harness.bpmn.

- **RTFM — the bootstrap docs**

The flow, with every term linked to the schema that defines it.

⚙ View

✎ Edit



Todos



STATUS

any status



PRIORITY

any priority



- The human-todos page still says 'Not built yet' — the store now exists

open

high

- who [github:litlfred](#)
- bean [folio-assistant-h32d](#) • pr [#336](#)
- pr [#314](#)

Pin



- Decide which roles content-pipeline-navigator and platform-boundary-guard take on

blocked

medium

- who [github:litlfred](#)
- bean [folio-assistant-29ij](#)
- bean [folio-assistant-h32d](#) • pr [#314](#)

Pin



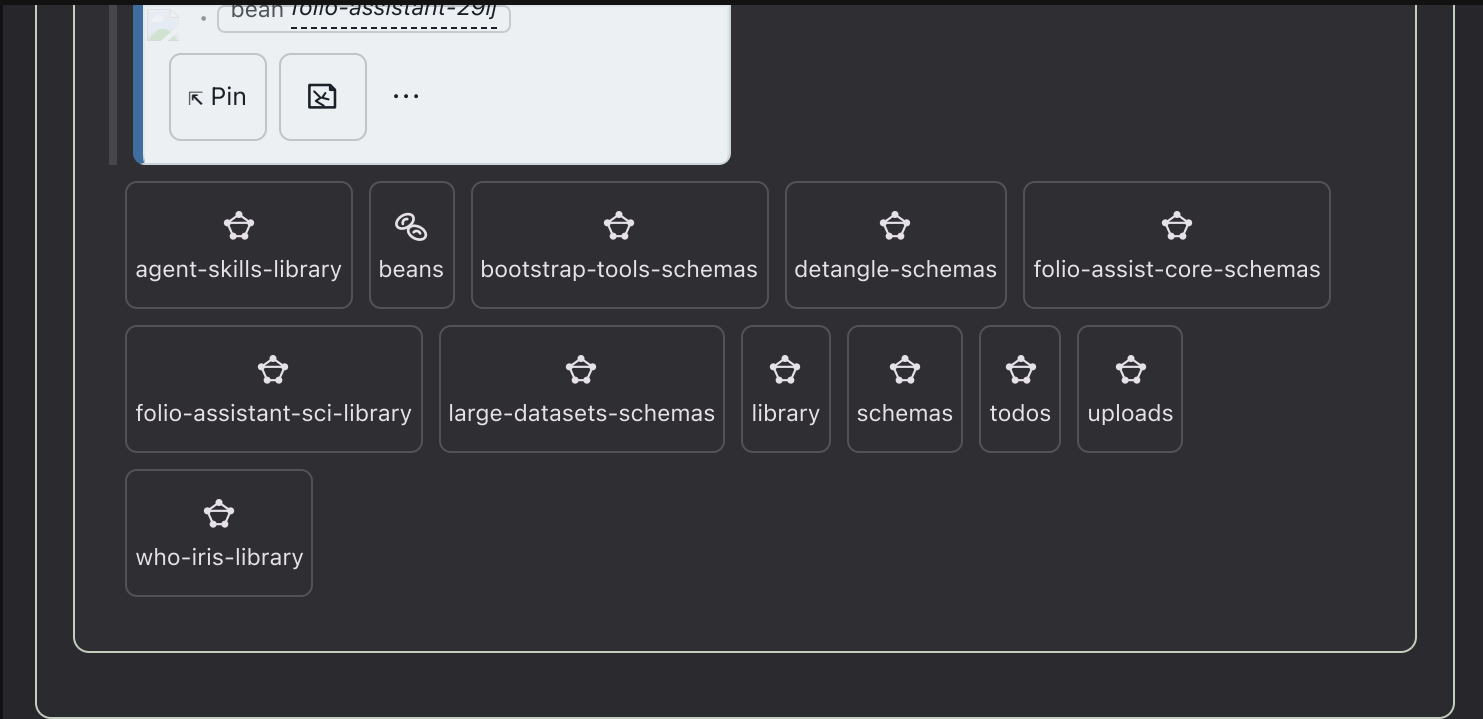
Process_Publication

- Decide what 'kick off' means mechanically for the two CI-watcher dispatch points

in_progress

medium

- process [Process_CodeReview](#)
- process [Process_Publication](#)
- who [github:litlfred](#)



folio-assistant

6/6 languages ✓ Swept 8/289 TR !

A content-agnostic agent skills framework for authoring rigorous content with a large language model — scientific papers and books, WHO SMART Guidelines, and FHIR Implementation Guides — backed by an MCP server, role-based access control, and a typed content-object model.

[Get started](#) [Install](#) [View on GitHub](#)

What each harness holds

One section per instantiated harness, in the same order as the dividers in the sidebar — the floor last.

[smart-trust](#)

WHO SMART Trust (smart.who.int.trust) v1.8.0 — the published IG's artefact index, reconstructed: 674 artefacts known, 19 materialized.

declared declared graph kinds:
directories:

visualisations you **0**
can open:

Nothing published to open yet.

Gaps in this instance's declaration (2)

[who-iris](#)

The WHO Institutional Repository for Information Sharing, catalogued by reference — 1,057,223 files known, three materialized.

declared **6** declared graph kinds: **6**
directories:

visualisations you **1**
can open:

- [library](#)

Gaps in this instance's declaration (3)

[folio-assistant](#)

The repository itself, acting as an initialized instance. NAMED `folio-assistant-checkout` rather than `folio-assistant`, which `cat-harness/harness.json` already uses: two declarations sharing a name makes `declaredBy` and a sticky's `contributedBy` ambiguous, so a directory can be attributed to the wrong instance while reading correctly. It declares the directories a person drops things into, which belong to the checkout rather than to any one layer inside it.

declared **1** declared graph kinds: **1**
directories:

visualisations you **0**
can open:

Nothing published to open yet.

Gaps in this instance's declaration (4)

[folio-assistant \(cat-harness\)](#)

computable adjudication and agentic test harness caaat-harness ca&at-harness .c&at-harness c@t-harness

declared 36 declared graph kinds: 22
directories:

visualisations you 8
can open:

• [beans](#) [health](#) [issue-marks](#) [library](#) [qa](#) [schemas](#) [todos](#) [uploads](#)

Gaps in this instance's declaration (2)

bootstrap

The graph an agent reads before it knows what this repository is.

declared 4 declared graph kinds: 3
directories:

visualisations you 0
can open:

Nothing published to open yet.

Gaps in this instance's declaration (1)

Four things, in order

1. The work plan is where you say what you are doing. Not a chat message, not a comment — [beans](#), a committed store any session or agent can read. Claim before you work so a sibling session does not pick up the same item; a bean that turns out not to be wanted is `scrapped`, with its reasons, never deleted.

```
scripts/install-beans.sh && export PATH="$HOME/.local/bin:$PATH"
beans list                                # what is open
beans create "<title>"                    # ...after checking the title does not exist
beans <id> --status in-progress           # claim it, visibly
```

2. Make your first folio. This repository is the *platform*; your content lives in its own. One command scaffolds it — the manifests, the declaration, the agent files and the link back here:

```
bun run init-folio --help
```

Then [Get started](#) walks the first block through validate, render and review.

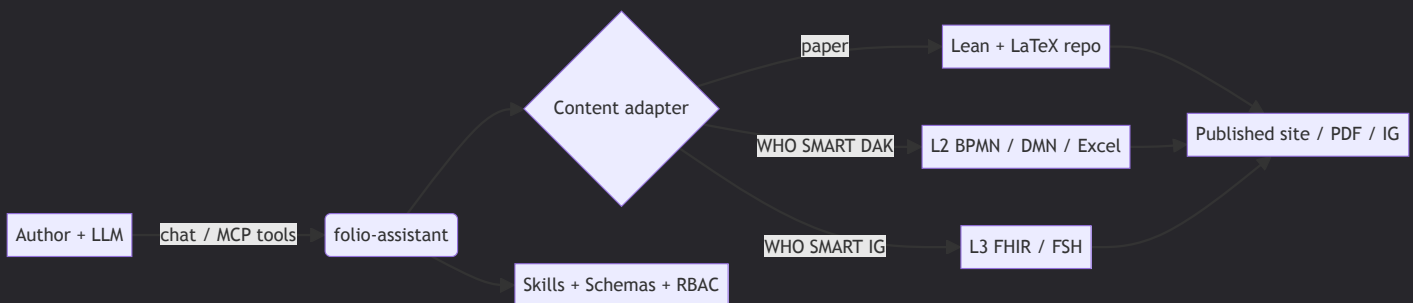
3. Know which kind of thing you are writing. A *document* is structured prose; a *paper* is that plus the block kinds whose assertion is a formal claim, backed by Lean and typeset through LaTeX. The choice decides which blocks are legal and which gates run: [Content types](#).

4. The documentation you will never read. [All of it](#) — the authoring guides, the architecture, the publication workflow, the generated schema and skill reference. It is here, it is thorough, and the honest expectation is that you will arrive at it from a search engine at the exact moment something breaks. That is a fine way to use it. The three steps above are the ones worth reading now.

What is folio-assistant?

folio-assistant is the *platform* — it does not contain content. It provides the skills, schemas, tooling, and an MCP (Model Context Protocol) server that an LLM-driven agent uses to plan, author, validate, review, test, and publish a **folio** of content that lives in a separate repository.

Separation of concerns. This documentation describes the *formalism of the framework* and *how to use folio-assistant* — deliberately kept **separate from any specific content**. Where content appears in these pages, it is purely illustrative (an *example*), never the canonical artifact.



Supported content types

folio-assistant is **pluggable** — each content type is handled by a content *adapter* and a matching skill *package*. The currently supported types:

Content type	Artifacts	Skill package
Scientific papers & books	Lean 4 formalization + LaTeX/Markdown	authoring-math
WHO SMART Guidelines DAKs	L2 artifacts — BPMN, DMN, Excel data dictionaries, personas	authoring-who-smart-guidelines
WHO SMART Implementation Guides	L3 FHIR resources, FSH, IG Publisher output	authoring-who-smart-guidelines
Others	Pluggable — add a new adapter + skill package	Adding a content type

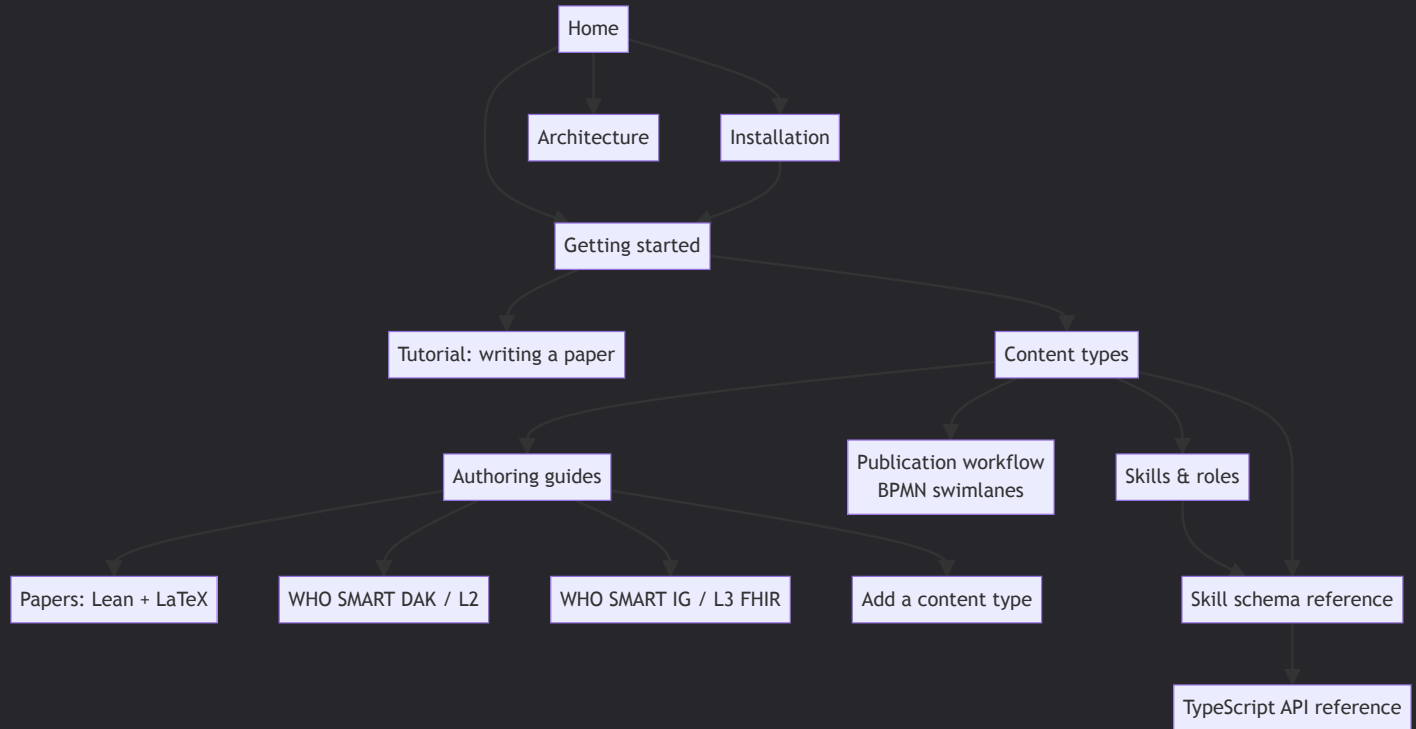
The cross-cutting [content-lifecycle](#) package (plan → author → validate → review → test → publish → feedback → retire) applies to every content type. The [publication workflow](#) models it properly — as BPMN swimlanes, with the roles, the HCI validation gate, and the shared work plan.

Where to go next

- [Installation](#) — prerequisites, clone, `bun install`, capability check.
- [Getting started](#) — connect the MCP server to your LLM and run your first skill.
- [Tutorial: Writing a paper with folio-assistant](#) — a full, LLM-driven walk-through with a mock chat session.
- [Content types](#) — the formalism of each authoring domain.
- [Publication workflow](#) — BPMN swimlane diagrams of the editing and publication processes: the HCI validation gate, who reviews what, and the shared work plan.
- [Agent onboarding](#) — orientation for an LLM agent dropped into a folio: first steps, finding skills, the content-object model, QA sidecars.
- [Skills & roles](#) — every skill and role, and how they work together with the LLM.
- [Skill schema reference](#) — generated input/output contracts for every skill.
- [TypeScript API reference](#) — the content-object model (`Block`, `Chapter`, `Paper`, builders, Zod constraints).
- [Architecture](#) — adapters, MCP server, RBAC, the block model.
- [The KGraph](#) — the subgraph taxonomy, which way the references run, and how repositories divide the work.
- [The Harness](#) — instantiation, the dependency walk, and what harnessing a directory obliges.

Two Skills are worth reading before the pages above, because everything else assumes them: [getting-started](#) routes what you are actually trying to do, and [placement](#) settles where a new node belongs before you make one.

Documentation map



The map nodes are clickable on the docs site.

▼ Linear listing (3)

Open notes (3)

Every note in this folio, in document order. This listing is the artefact; the board is an overlay over it.

1

The human-todos page still says
'Not built yet' — the store now exists

Status: open

Priority: high

Raised by: agent

Created: 2026-09-19

2

Decide which roles content-pipeline-
navigator and platform-boundary-
guard take on

Status: blocked

Priority: medium

Raised by: agent

Created: 2026-09-19

About: [beans-and-todos > human-todos](#)

`content/docs/beans-and-todos/human-todos.md` opens `***Not built yet***`, and that is no longer true. `todos/` is a declared graph with items on disk, a reader, a published JSON index and a board on the site.

This todo is attached to that block, so it renders beside the sentence it contradicts. That is the per-page assignment working on a real case rather than a demonstration — the page is genuinely stale and somebody has to decide what it should say now.

What the page still gets RIGHT, and must not lose

Three of its four claims survive, and they are the valuable part:

- `***Do not repurpose beans for it.***` Still correct, and now structural: `beans/` and `todos/` are separate declared graphs. - `***When a todo is shown.***` Still undecided. This PR answers it for the navbar and the board; it does not answer whether an open item should surface unasked, and the page's objection stands — surfacing every item on every turn is noise, surfacing none is a list nobody reads. - `***How one is retired.***` Still undecided. `TodoStatus` carries `resolved` and `wontfix`, but nothing retires one, and the page's point holds: a person abandoning a reminder is ordinary and needs no ceremony, so the bean rules do not transfer.

What has changed and needs writing down

- `***How a person creates one***` is answered in the worst way: by an agent writing a file. There is no human path at all yet. - The 2x2 the page is missing — memory against workflow management, human against agent — with the empty quadrant named.

Done when

The page says what is true. Cheapest correct move is an edit to that block, not a rewrite: strike "Not built yet", keep the three open questions, add what now exists and who created it.

About: [publication-workflow > agents-and-system-actors](#)

`ci-health-watcher` is settled: it is a mechanical participant in the CI process, so it takes `build-pipeline` and `validation-pipeline`, both `actorKind: "system"`. Declared, and `memoryForRoles` now discriminates because of it.

`***The other two are not settled by that argument, and the reason is precise.***` Both system roles are described as running a fixed program and exercising `*no judgement*`. That is exactly what makes them the right home for a watcher — and exactly why they are the wrong home for these two:

- `***platform-boundary-guard***` decides whether a literal naming one folio belongs in platform code, and whether a new content type needs an adapter or only a profile. Those are judgements; a wrong one is not a failed check. - `***content-pipeline-navigator***` is closer to mechanical — it knows which stage owns which file — but it also answers "should this be a block kind", which is not.

What differs depending on the answer

Nothing breaks either way today: memory is scoped by agent and the duplication is already fixed, because one node can reach several agents. What changes is whether a fact written for one of these two reaches an agent standing in that lane, or only the agent named in the file.

Options

- `***Leave both agent-scoped, and say so in the role model.***` Costs nothing, and is honest — neither is a lane anyone else acts in. The cost is that the role axis stays half-used, with one actor on it.
- `***Give them a judgement-bearing existing role***` — `editor` and `code-reviewer` are the nearest. Cheap, and wrong if the fit is loose: a forced edge is one every later audit trusts as true.
- `***Add roles for them.***` Honest edges. But `AGENTS.md` warns under `skill-in-role-or-process` against "inventing roles and activities to absorb tools that do not want them", and a role

Every todo carries a pencil to exactly this file, so the edit is one click from where the stale sentence renders.

[View source](#) · [Edit](#)

- [who: github:litlfred](#) [bean: folio-assistant-h32d](#)
- [PR: #336](#) [PR: #314](#)

with one actor describes the tool rather than the work.

No recommendation. The `ci-health-watcher` case was settled by a property of the work (it exercises no judgement); neither option here has an equivalent, which is why it is a person's call and not an agent's.

If nothing is decided

Option 1 is the status quo and nothing regresses.

[View source](#) · [Edit](#)

- [who: github:litlfred](#) [bean: folio-assistant-29ij](#)
- [bean: folio-assistant-h32d](#) [PR: #314](#)

3

Decide what 'kick off' means mechanically for the two CI-watcher dispatch points

Status: in_progress

Priority: medium

Raised by: agent

Created: 2026-09-19

About: [publication-workflow > agents-and-system-actors](#)

Stated 2026-09-19: a CI watcher `"should kick off during feature development once changes are made to feature branch. and when approved for final publication/merge to main."`

The two dispatch points are clear. `**What performs the dispatch is not**`, and the three candidates differ enough that guessing would be wasted work.

ANSWERED, 2026-09-19

Owner: `"kick off if like task or workflow initiation or bean roast"`.

`**Option 2.**` A dispatch point is a `**task / workflow initiation**`, and it opens a `**bean**` — not a GitHub Actions job. That is the answer the repo's own machinery is already shaped for: an activity carrying `<folio:bean op="claim"/>`

performs the bean operation when the step completes, so "start the watcher" and "record that it is running" are one declared step rather than two mechanisms that can disagree.

It also settles the cost I flagged against option 2 — that a BPMN activity only fires while somebody is working. With a bean opened at initiation, the work is **durable**: a later session sees the bean whether or not the session that opened it is still alive.

Remaining, and now narrow: which diagrams host the two steps, and whether the feature-branch one needs a process that does not yet exist.

Options, with what each costs

1. ***A GitHub Actions job.*** Fires without anybody present, which is the whole premise of `xom7` — the failure being guarded against is a quiet stretch with nobody looking. Cost: it cannot read the session's memory, so the "dispatch an agent with a specific memory context" half does not happen.
2. ***An activity in a BPMN process***, performed by a session. Gets the memory context, and puts the dispatch where `workflow\_next` can report it. Cost: it only fires while somebody is working — precisely the stretch that is already covered.
3. ***An MCP tool a workflow step calls.*** Both, at the price of building the tool and deciding who calls it.

Where the two dispatch points already live

Both are steps in diagrams that exist, which is why this todo is tagged with them rather than describing them:

- ***"approved for final publication / merge to `main`"* is `Process\_Publication` (`draft-to-publication.bpmn`), whose lane carries **Authorise the release** and **Version, tag and publish**. The dispatch point the owner named is that authorisation step. - ***"once changes are made to feature branch"* is `Process\_CodeReview`, reached from `Process\_Review`.

What is already true, so this is not started from nothing

The two dispatch points are process events, so wherever they are performed they should be

DRAWN in BPMN under `skills/workflows/`. The likely homes are `draft-to-publication` for the merge-to-main gate and a feature-branch process that may not exist yet — that needs checking rather than assuming.

If nothing is decided

Nothing fires. `bun run check:ci-health` still runs in the session-start sweep and `ci-health.yml` still runs weekly, so the watcher is not silent — it is just not tied to the two moments named above.

[View source](#) · [Edit](#)

- process: Process_CodeReview
- process: Process_Publication [who: github:litlfred](#)
- bean: folio-assistant-29ij

folio-assistant — code under the Apache License 2.0, documentation under CC BY 3.0. Generated schema and API references are rebuilt from source on every push to main.

deployed [dev](#)

☐ ≡ Keep navigation open

• HARNESSES 5

- smart-trust
- who-iris
- folio-assistant
- folio-assistant (cat-harness)
- bootstrap

• DEPENDENCIES 9